

Reconocimiento de gestos de mano con MediaPipe y una CNN

Guía del trabajo realizado en `HAR_mediapipe`: datos, entrenamiento, resultados, cómo cargar el modelo y cómo lanzar la demo con cámara.

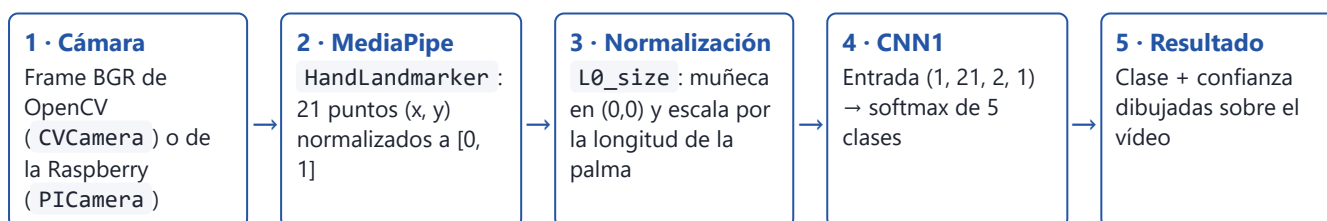
Asignatura	PIDS · Curso 4
Grupo / datasets	Gonzalo, Iria, Marta y Miguel (un dataset por persona)
Gestos (5 clases)	<code>OK</code> <code>dislike</code> <code>guerra</code> <code>like</code> <code>paz</code>
Modelo recomendado	<code>HAR_mediapipe/models/pids_gestures_CNN1_L0_size.keras</code>
Precisión	98,98 % en test · 98,28 % dejando fuera a una persona (LOPO)

Contenido:

1. Visión general del sistema
2. Qué hemos hecho
3. Resultados
4. Estructura de ficheros
5. Preparar el entorno
6. Grabar un dataset
7. Extraer landmarks
8. Entrenar el modelo
9. Demo en directo con la cámara
10. Cargar el modelo desde tu propio código
11. Detalles técnicos
12. Problemas frecuentes y pendientes

1. Visión general del sistema

El sistema no clasifica la imagen directamente. Primero MediaPipe localiza los **21 puntos clave (landmarks) de la mano** y después una red neuronal muy pequeña clasifica esas 42 coordenadas. Así el modelo es ligero, se entrena en segundos y apenas le afectan el fondo, la iluminación o la ropa.



Para entrenar se sigue el mismo camino, pero con las imágenes guardadas en disco:

```
record_dataset.py → extract_landmarks.py → train_gestures.py → demo-custom-dataset.py  
(imágenes .jpg)      (landmarks .csv/.npy)      (.keras + .meta.json)      (cámara en directo)
```

2. Qué hemos hecho

2.1 Grabación de cuatro datasets

- Cada integrante grabó su propio dataset con `record_dataset.py`: `dataset_pids_gonzalo`, `_iria`, `_marta` y `_miguel`.
- **50 imágenes por gesto** (1 imagen por segundo), repartidas **70 % / 30 %**: 35 para entrenar y 15 para test. En total, 1000 imágenes.

- MediaPipe encontró la mano en **991 de 1000 imágenes**. Las 9 imágenes sin detección quedan anotadas en `HAR_mediapipe/logs.txt` y no se usan.

2.2 Nuevo script de extracción: `extract_landmarks.py`

Antes la extracción se hacía a mano desde el notebook. Ahora hay un script que procesa todos los `dataset_pids_*` de una vez (o solo los que le indiques). También comprueba que train y test tengan las mismas clases y muestra un resumen de las detecciones.

2.3 Corrección y mejora de la normalización (`landmarksLib.py`)

- **Error corregido** en `normalize_from_0_landmark`: el bucle empezaba en el índice 2, así que el propio landmark 0 (la muñeca) nunca se trasladaba a (0, 0). Ahora empieza en 0.
- **Nueva normalización de tamaño** `normalize_size_from_palm`: escala cada mano para que la distancia muñeca (landmark 0) → nudillo del dedo corazón (landmark 9) valga 1. La longitud de la palma apenas cambia entre gestos, así que esto elimina el efecto de **la distancia a la cámara** y del **tamaño de la mano de cada persona**.
- Nueva función `NormalizeLandmarks(data, norm_type)` con tres modos: `None`, `L0` y `L0_size`. Tanto el entrenamiento como la demo la usan, así que el preprocesado es el mismo en los dos.

2.4 Nuevo script de entrenamiento: `train_gestures.py`

- Junta los cuatro datasets y entrena una red `CNN1` o `CNN2` con la normalización elegida.
- **Early stopping con validación propia**: se aparta un 15 % del conjunto de *train* para validar. El test no se usa nunca para elegir los pesos, así que la precisión en test no sale inflada.
- Hay dos modos de evaluación:
 - `split`: entrena con todos los *train*, evalúa con todos los *test* y **guarda el modelo**.
 - `lopo` (*leave-one-person-out*): entrena con 3 personas y prueba con la 4ª, rotando entre las cuatro. Mide si el modelo funciona con **alguien que no ha visto nunca**, que es lo que pasa en la demo. En este modo no se guarda ningún modelo.
- Junto a cada modelo guarda un `.meta.json` con el orden de las clases y la normalización usada. Los resultados se añaden a `HAR_mediapipe/data/results/results.txt`.

2.5 Arreglos en la demo (`demo-custom-dataset.py`)

- La ruta apuntaba a `FER_mediapipe/src` (el proyecto de emociones). Ahora apunta a `HAR_mediapipe/src` e importa `landmarksLib`.
- Las clases ya no se escriben a mano en el script: se leen del `.meta.json` del modelo, junto con la normalización.
- Antes de predecir, se aplica a cada frame **la misma normalización** que en el entrenamiento. Antes se pasaban las coordenadas sin normalizar a la red.

3. Resultados

Precisión con intervalo de confianza del 95 %. **Split** = 295 imágenes de test de las 4 personas. **LOPO** = las 991 imágenes, cada persona evaluada con un modelo que no la ha visto.

Red	Normalización	Split: precisión	Split: F1 macro	LOPO: precisión	LOPO: peor persona
CNN1	None	98,98 ± 1,14	0,9899	95,96 ± 1,23	86,23 (Marta)
CNN1	L0	98,64 ± 1,32	0,9864	96,37 ± 1,16	89,07 (Marta)
CNN1	L0_size	98,98 ± 1,14	0,9898	98,28 ± 0,81	97,57 (Marta)
CNN2	L0_size	98,31 ± 1,47	0,9830	97,98 ± 0,88	95,95 (Marta)

LOPO por persona (CNN1)	None	L0	L0_size
Gonzalo	97,98	98,79	97,98
Iria	99,60	97,98	98,38
Marta	86,23	89,07	97,57
Miguel	100,00	99,60	99,20

Conclusión

Con el split normal todas las configuraciones parecen iguales ($\approx 99\%$), porque en test aparece la misma persona que en train. La diferencia se ve en LOPO: sin normalizar, el modelo falla bastante con Marta (86 %). Con `L0_size` sube al 97,6 % y el resultado es parecido para las cuatro personas. Por eso **CNN1 + L0_size** es el modelo que usa la demo. Además es el más rápido de entrenar (17,5 s) y solo tiene 21.797 parámetros.

4. Estructura de ficheros

```

IE-workspace/
├── common/
│   ├── cameras.py          CVCamera (OpenCV/V4L2), PICAmer (Raspberry), CameraConfig
│   ├── gui.py              Colors, WindowMessage, wait_for_keypress
│   └── evaluation.py        CalculateCI (intervalo de confianza 95 %)
├── HAR_mediapipe/
│   ├── data/
│   │   ├── dataset_pids_<persona>/
│   │   │   ├── train/<clase>/*.jpg    imágenes grabadas (35 por clase)
│   │   │   ├── test/<clase>/*.jpg     imágenes grabadas (15 por clase)
│   │   │   ├── landmarks/{train,test}/*_poses_landmarks.csv
│   │   │   ├── {train,test}_classes_list.txt
│   │   │   ├── {train,test}_dataset_with_labels.csv
│   │   │   └── {train,test}_dataset.npy / _labels.csv
│   │   └── results/results.txt        histórico de entrenamientos
│   ├── models/
│   │   ├── hand_landmarker.task       modelo de MediaPipe para detectar la mano
│   │   ├── pids_gestures_CNN1_L0_size.keras    ★ modelo final
│   │   ├── pids_gestures_CNN1_L0_size.meta.json  clases + normalización
│   │   └── pids_gestures_{CNN1_None, CNN1_L0, CNN2_L0_size}.*
│   └── src/
│       ├── config.py                Config y ConfigMediapipeDetector
│       ├── landmarksLib.py           extracción, NormalizeLandmarks, ArrangeInputDataForNetwork
│       ├── record_dataset.py         grabar dataset con la cámara
│       ├── extract_landmarks.py      imágenes → landmarks
│       ├── train_gestures.py         entrenar y evaluar
│       └── demo-custom-dataset.py    demo en directo con cámara

```

5. Preparar el entorno

Los scripts hay que ejecutarlos en **Linux** (en nuestro caso WSL2 con Ubuntu, o la Raspberry Pi). `common/gui.py` usa `termios`, que no existe en Windows, así que el `venv env/` de Windows **no sirve** para estos scripts.

Usar el entorno que ya está creado

```
cd "/mnt/c/Users/34619/Desktop/Universidad/Curso 4/PIDS/IE-workspace"
source ~/venvs/ie-workspace/bin/activate      # Python 3.12, keras 3.15.1, mediapipe 1.0.1
```

O crear uno nuevo

```
python3 -m venv ~/venvs/ie-workspace
source ~/venvs/ie-workspace/bin/activate
pip install -r requirements.txt
```

Cámara en WSL2

WSL no ve la webcam por defecto. Hay que pasarla con `usbipd` desde PowerShell como administrador (`usbipd list`, `usbipd bind --busid X-Y`, `usbipd attach --wsl --busid X-Y`). Después comprueba en WSL que existe `/dev/video0` y que tu usuario está en el grupo `video` (`groups`).

6. Grabar un dataset (`record_dataset.py`)

Antes de grabar, edita el bloque `Config(...)` del script:

```
config = Config(classes=['OK', 'dislike', 'guerra', 'like', 'paz'],
                 dataset_dir='HAR_mediapipe/data/dataset_pids_<nombre>/',    # uno por persona
                 num_images_per_class=50,
                 training_percentage=70,
                 use_landmarks=True)
```

```
python3 HAR_mediapipe/src/record_dataset.py
```

1. Para cada clase aparece una pantalla de preparación con los landmarks dibujados. Pon el gesto y pulsa `s` para empezar (`q` para salir).
2. Guarda 1 imagen por segundo hasta llegar a 50. Mueve un poco la mano (distancia, giro, posición) para que el dataset sea variado.
3. Las primeras 35 van a `train/<clase>/` y las 15 últimas a `test/<clase>/`.

Cuidado

Los nombres de fichero siempre empiezan en `<clase>_00001.jpg`. Si vuelves a grabar en la misma carpeta, **se sobrescriben las imágenes anteriores**. Usa un `dataset_dir` nuevo para cada persona o sesión.

7. Extraer landmarks (`extract_landmarks.py`)

```
# Todos los dataset_pids_*
python3 HAR_mediapipe/src/extract_landmarks.py

# Solo algunos, guardando también las imágenes con los landmarks dibujados
python3 HAR_mediapipe/src/extract_landmarks.py dataset_pids_gonzalo dataset_pids_iria --save-images
```

Crea dentro de cada dataset los `.csv` / `.npy` de la sección 4. Al terminar muestra cuántas manos se han detectado por clase. Las etiquetas numéricas siguen el **orden alfabético** de las carpetas (línea n de `train_classes_list.txt` = etiqueta n).

8. Entrenar el modelo (`train_gestures.py`)

```
# Entrena el modelo final y lo guarda (.keras + .meta.json)
python3 HAR_mediapipe/src/train_gestures.py --network CNN1 --norm L0_size

# Evaluación leave-one-person-out (no guarda modelo)
python3 HAR_mediapipe/src/train_gestures.py --network CNN1 --norm L0_size --eval lopo

# Solo con algunos datasets
python3 HAR_mediapipe/src/train_gestures.py dataset_pids_gonzalo dataset_pids_marta
```

Opción	Valores	Por defecto
<code>datasets</code>	nombres de carpeta dentro de <code>HAR_mediapipe/data/</code>	todos los <code>dataset_pids_*</code>
<code>--network</code>	<code>CNN1</code> , <code>CNN2</code>	<code>CNN1</code>
<code>--norm</code>	<code>None</code> , <code>L0</code> , <code>L0_size</code>	<code>L0_size</code>
<code>--eval</code>	<code>split</code> (guarda modelo), <code>lopo</code>	<code>split</code>

Hiperparámetros (constantes al principio del script): AdamW (lr = 0,001, weight decay = 1e-4), batch 32, hasta 300 épocas, early stopping con paciencia 50 sobre `val_loss` , dropout 0,3 y semilla 2022. El modelo se guarda como `HAR_mediapipe/models/pids_gestures_<red>_<norm>.keras` .

9. Demo en directo con la cámara (`demo-custom-dataset.py`)

```
cd "/mnt/c/Users/34619/Desktop/Universidad/Curso 4/PIDS/IE-workspace"
source ~/venvs/ie-workspace/bin/activate
python3 HAR_mediapipe/src/demo-custom-dataset.py
```

Se abre una ventana con el vídeo, los landmarks de la mano y el texto *Predicted class: <gesto> (confianza)* en un color distinto por clase. Si no hay mano aparece `None` . La predicción se actualiza cada 0,25 s. Pulsa `q` para salir.

Qué puedes cambiar en el script

Variable	Para qué sirve
MODEL_PATH	Modelo a usar. Si pones otro <code>pids_gestures_*.keras</code> , las clases y la normalización se cogen automáticamente de su <code>.meta.json</code> .
index_cam	En <code>CVCamera(..., index_cam=0)</code> : 0 = primera cámara (<code>/dev/video0</code>). Prueba 1, 2... si usas otra.
CameraConfig(FPS, resolution)	'highres' 1280×720, 'large' 640×480, 'small' 320×200. Baja la resolución si va lento.
ON_RASPBERRY_PI	True para usar <code>PICamera</code> (<code>picamera2</code>) en lugar de OpenCV.
ON_SENSE_HAT	True para inicializar la SenseHAT en la Raspberry.
debug_HAR	True para ver por consola los landmarks y la confianza de cada predicción.

Qué hace el bucle principal

```
image = cam.read_frame() # frame BGR
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # MediaPipe espera RGB
detection_result = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=image_rgb))
if detection_result.hand_landmarks:
    image, landmark_values = draw_landmarks_on_image(image, detection_result)
    x = np.reshape(np.array(landmark_values), (1, 42)) # x0,y0,x1,y1,...
    x = NormalizeLandmarks(x, norm_type) # misma normalización que en train
    x = ArrangeInputDataForNetwork(x) # → (1, 21, 2, 1)
    predictions = model(x)
    pred = classes[np.argmax(predictions)]
```

10. Cargar el modelo desde tu propio código

Ejemplo mínimo e independiente de la demo. **Lo hemos probado** con el venv del proyecto y clasifica correctamente una imagen de test de cada gesto (confianza > 0,99). Ejecútalo desde la raíz del repositorio.

```

import json, os, sys
import cv2
import keras
import mediapipe as mp
import numpy as np

sys.path.append(os.path.join(os.getcwd(), "HAR_mediapipe", "src"))
sys.path.append(os.path.join(os.getcwd(), "common"))
from config import ConfigMediapipeDetector
from landmarksLib import NormalizeLandmarks, ArrangeInputDataForNetwork

MODEL_PATH = "HAR_mediapipe/models/pids_gestures_CNN1_L0_size.keras"
model = keras.models.load_model(MODEL_PATH)
with open(MODEL_PATH.replace(".keras", ".meta.json")) as f:
    meta = json.load(f) # {"classes": [...], "norm_type": "L0_size", ...}
detector = ConfigMediapipeDetector("HAR_mediapipe/models/hand_landmarker.task")

def predecir_gesto(image_bgr):
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    result = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=image_rgb))
    if not result.hand_landmarks:
        return None, 0.0 # no hay mano
    puntos = [[lm.x, lm.y] for lm in result.hand_landmarks[0]]
    x = np.array(puntos).reshape(1, 42)
    x = NormalizeLandmarks(x, meta["norm_type"])
    x = ArrangeInputDataForNetwork(x)
    probs = model.predict(x, verbose=0)[0]
    return meta["classes"][int(np.argmax(probs))], float(np.max(probs))

# Con una imagen de disco
img = cv2.imread("HAR_mediapipe/data/dataset_pids_gonzalo/test/like/like_00041.jpg")
print(predecir_gesto(img)) # ('like', 0.9996)

# Con la webcam (sin ventana ni dibujos)
cap = cv2.VideoCapture(0)
while True:
    ok, frame = cap.read()
    if not ok:
        break
    print(predecir_gesto(frame))

```

Las tres reglas para que las predicciones tengan sentido

1. **BGR → RGB** antes de pasar el frame a MediaPipe.
2. **La misma normalización** que en el entrenamiento: léela siempre del `.meta.json` y no la pongas a mano.
3. **El mismo orden de clases** que en el entrenamiento (alfabético: OK, dislike, guerra, like, paz), también sacado del `.meta.json`.

11. Detalles técnicos

Formato de los datos

- MediaPipe devuelve 21 landmarks con `x`, `y` en `[0, 1]` respecto al tamaño de la imagen. No usamos `z`.
- Se guardan aplanados como `x0, y0, x1, y1, ..., x20, y20` (42 valores).
- `ArrangeInputDataForNetwork` los convierte a `(N, 21, 2, 1)`: fila = landmark, columna = x/y.

Normalización `L0_size`

- Se resta la muñeca (landmark 0) a todos los puntos.
- Se divide por la longitud de la palma, $\|p_9 - p_0\|$.

El resultado no depende de dónde esté la mano en la imagen ni de lo cerca que esté de la cámara.

Arquitecturas

CNN1 (21.797 parámetros)

Conv2D 16 filtros (5×1), ReLU
Dropout 0,3
Flatten (672)
Dense 32, ReLU
Dropout 0,3
Dense 5, softmax

CNN2 (5.477 parámetros)

Conv2D 16 filtros (3×3), sigmoid
MaxPooling 2×2
Flatten (160)
Dense 32, ReLU
Dropout 0,3
Dense 5, softmax

El filtro 5×1 de CNN1 recorre landmarks consecutivos, que en MediaPipe van agrupados por dedos (1–4 pulgar, 5–8 índice...).

12. Problemas frecuentes y pendientes

Síntoma	Causa / solución
<code>ModuleNotFoundError: landmarksLib</code> o no encuentra <code>hand_landmarker.task</code>	Las rutas son relativas a la raíz del repositorio . Haz <code>cd IE-workspace</code> antes de ejecutar.
No module named 'termios'	Estás usando Python de Windows (<code>env/</code>). Usa el venv de Linux/WSL (sección 5).
Ventana en negro o <i>Waiting for camera input</i> sin parar	La cámara no llega a WSL (<code>usbipd attach</code>), el <code>index_cam</code> no es el correcto o falta el grupo <code>video</code> . CVCamera usa el backend V4L2, que solo existe en Linux.
Siempre predice la misma clase, o con poca confianza	No coinciden la normalización o el orden de las clases con los del entrenamiento. Usa el <code>.meta.json</code> . Revisa también la iluminación y que se vea la mano entera.
Va lento	Baja la resolución a <code>'large'</code> o <code>'small'</code> en <code>CameraConfig</code> .

Detalles pendientes detectados en la demo

- `num_frames+=1` asigna 1 en lugar de sumar (debería ser `num_frames += 1`). Solo afecta a los mensajes de depuración.
- Con `ON_RASPBERRY_PI = True` y `ON_SENSE_HAT = False`, al pulsar `q` se llama a `sense_hat.clear()` sin que `sense_hat` exista, y el script termina con un error. Habría que comprobar `ON_SENSE_HAT` antes de esa

llamada.

- La SenseHAT se inicializa, pero la demo todavía no muestra en ella ningún icono del gesto reconocido.